

```
File Edit View Bookmarks Settings Help
$ g++ tcp_server.cc -o server
$ ./server
Message Received from Client : Hello OSFY Friends
```

Figure 2: TCP server terminal

```
File Edit View Bookmarks Settings Help
$ g++ tcp_client.cc -o client
$ ./client

Enter the Message: Hello OSFY Friends
Acknowledgement from Server : Your Message Received
```

Figure 3: TCP client terminal

```
struct hostent *server;
char buffer[256];
portno = 3333;
sockfd = socket(AF_INET, SOCK_STREAM, 0);
server = gethostbyname("127.0.0.1");
bzero((char *) &serv_addr, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
bcopy((char *)server->h_addr, (char *)&serv_addr.sin_
addr.s_addr, server->h_length);
serv_addr.sin_port = htons(portno);
connect(sockfd, (struct sockaddr *) &serv_
addr, sizeof(serv_addr));
cout<<"\nEnter the Message: ";
bzero(buffer, 256);
cin.getline(buffer, 256);
n = send(sockfd, buffer, strlen(buffer), 0);
bzero(buffer, 256);
n = read(sockfd, buffer, 255);
cout<<"Acknowledgement from Server : "<<buffer<<"\n\n";
close(sockfd);
return 0;
}
```

This is a very simple client-server program in which the client sends a message to the server, and the server prints the message on its terminal before replying with an acknowledgement. The client, in turn, prints the acknowledgment from the server on its terminal. The function *bcopy* copies memory regions of arbitrary length. The client is also assigned the port 3333, same as the server. The IP address used by the client is 127.0.0.1, which is a special IPv4 address denoting the local host. This happens because the client and server processes are running on the same system. But if the server is running on a system other than the client system, then use the IP address of the system running the server process in the client program. Since TCP is a connection-oriented protocol, the server program should be executed before the client program; otherwise, the client will not be able to connect to the server. Figure 3 shows the terminal executing the client program.

An example of UDP peer-to-peer communication

Usually, in UDP, the communicating nodes are not considered as client and server, but are treated as peers involved in communication. To save some space, I haven't included the program here. You can download all the programs discussed in this article from opensourceforu.com/article_source_code/june2015/ns3.zip.

There are two UDP based programs — *udp_sender.cc* and *udp_receiver.cc*. But there are no big differences between TCP and UDP programs. Instead of using *send* and *recv* functions, UDP programs use *sendto* and *recvfrom* functions. Unlike TCP, in UDP it is not mandatory for the receiver process to be active before the sender process. Each packet contains the destination address in UDP because it is a connectionless service. This is not required in TCP. While using UDP in programs the functions *listen*, *accept* and *connect* are not used. Even though the programs are somewhat similar, how TCP and UDP actually work is quite different. But we will deal with those differences while learning ns-3.

Socket programming contains a lot of topics which countless authors and textbooks are trying to tackle effectively. To summarise everything about socket programming into a few pages, I was compelled to omit many important topics. I know crimes of omission are far more serious than crimes of commission, but do forgive me. For a thorough understanding of socket programming, do refer to the book titled 'UNIX Network Programming, Volume 1: The Sockets Networking AP' by Richard Stevens.

Maintaining ns-3 with Mercurial

All large software projects require a revision control tool for source code management during their development phase, and ns-3 is no exception. The very popular revision control tool called Mercurial is used with ns-3. But I will not discuss Mercurial in this series, because its need arises only when you start contributing code to the ns-3 project. At present, our aim is to familiarise ourselves with ns-3; becoming a part of ns-3 development is not our goal. If at a later stage, your mastery over ns-3 allows you to contribute code, then an understanding of Mercurial will be a necessity. There are Mercurial commands like *clone*, *add*, *commit*, etc, which will allow you to make a copy of an existing project and add your own content to it. I will be content if I could help somebody reach that level with this series on ns-3. 

By: Deepu Benson

The author has nearly 15 years of programming experience. He currently works as assistant professor in Amal Jyothi College of Engineering, Kerala. He maintains a technical blog at www.computingforbeginners.blogspot.in. He can be reached at deepumb@hotmail.com.